Instructions

Close ×

Tasks

In this lab, you will optimize the performance of a matrix multiplication implementation by applying various loop transformation techniques and leveraging OpenMP Parallelization through compiler flags. Use this GCC documentation as a reference for optimization controls.

The Matrix Multiplication Algorithm

You are provided with a C implementation of the matrix multiplication algorithm. The program multiplies two square matrices, A and B, of size N x N, and stores the result in matrix C. The matrix multiplication operation is defined as: C = A × B.

Part 1: Baseline Implementation

Run the shell script (`optimize.sh`) so that it compiles the provided `matrix_multiplication.c` file. The shell script will run and measure and record the execution time for this baseline implementation using the predefined function `run_and_time`.

Part 2: Optimizations

Modify your shell script (`optimize.sh`) so that it compiles `matrix_multiplication.c` with various optimization flags provided by the GCC compiler to enable different optimization techniques:

1. Loop Interchange: Compile the code with the flag, which enables loop interchange optimization. This optimization changes the order of the nested loops to improve memory access patterns and cache utilization.
2. Loop Tiling: Compile the code with the flag, which enables loop tiling (also known as loop blocking) optimization. This optimization divides the loop iterations into smaller "tiles" or blocks to improve cache utilization and reduce cache misses.
3. Loop Vectorization: Compile the code with the flag, which enables loop vectorization. This optimization takes advantage of SIMD (Single Instruction Multiple Data) instructions available on modern CPUs.
4. OpenMP Parallelization: Compile the code with the flag, which enables OpenMP (Open Multi-Processing) parallelization. This optimization allows the compiler to automatically parallelize loops and distribute the work across multiple CPU cores, potentially reducing execution time on multi-core systems.

Measure and record the execution time for each technique, and compare it with the baseline implementation. Analyze the performance differences from each optimization.

Submission

After analyzing all optimizations, submit `optimize.sh` for grading. Your shells script should contain the baseline implementation and the four optimization techniques.

Your shell script will be graded using the following criteria:

1. Your script compiles the code with the loop interchange flag (1 point)
2. Your script compiles the code with the loop tiling flag (1 point)
3. Your script compiles the code with the loop vectorization flag (1 point)
4. Your script compiles the code with the OpenMP parallelization flag (1 point)

Mark as completed